

# Bài 01: Tạo MYSQL Stored Procedure đầu tiên

Trong bài chúng ta sẽ viết một Procedure trả về danh sách sản phẩm của bảng Products, chính vì vậy trước tiên bạn phải tạo một bảng tên là Products.

```
CREATE TABLE IF NOT EXISTS `products` (  
  `id` INT(11) NOT NULL AUTO_INCREMENT,  
  `title` VARCHAR(255) COLLATE utf8_unicode_ci DEFAULT NULL,  
  `content` TEXT COLLATE utf8_unicode_ci,  
  PRIMARY KEY (`id`)  
) ENGINE=INNODB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci AUTO_INCREMENT=3  
;  
  
--  
-- Contenu de la table `products`  
--  
  
INSERT INTO `products` (`id`, `title`, `content`) VALUES  
(1, 'Học lập trình online tại freetuts.net', 'Gioi thieu website Học lập  
trình online tại freetuts.net'),  
(2, 'Tutorials học Stored Procedure', 'Website Tutorials học Stored  
Procedure');
```

Chúng ta sẽ viết một Procedure với tên là `GetAllProducts()`, nhiệm vụ của thủ tục này là truy vấn lấy danh sách tất cả sản phẩm nằm trong bảng Products. Nhưng trước tiên chúng ta tìm hiểu cú pháp khai báo tạo mới một Procedure như sau:

```
1 DELIMITER $$  
2 CREATE PROCEDURE procedureName()  
3 BEGIN  
4   /*Xu ly*/  
5 END; $$  
6 DELIMITER ;
```

**Trong đó:**

- Dòng đầu tiên `DELIMITER $$` dùng để phân cách bộ nhớ lưu trữ thủ tục Cache và mở ra một ô lưu trữ mới. Đây là cú pháp nên bắt buộc bạn phải nhập như vậy
- Dòng `CREATE PROCEDURE procedureName()` dùng để khai báo tạo một Procedure mới, trong đó `procedureName` chính là tên thủ tục còn hai từ đầu là từ khóa.
- `BEGIN` và `END; $$` dùng để khai báo bắt đầu của Procedure và kết thúc Procedure
- Cuối cùng là đóng lại ô lưu trữ `DELIMITER ;`
- Bây giờ để tạo mới một Procedure với tên là `GetAllProduct` thì chúng ta sẽ làm như sau:

```
1 DELIMITER $$  
2 CREATE PROCEDURE GetAllProducts()  
3 BEGIN  
4   /*Xu ly*/  
5 END; $$
```

```
5 DELIMITER ;
```

```
6
```

- Sau đó bạn chạy câu SQL này và nó báo thành công tức là bạn đã tạo mới một thủ tục với tên là GetAllProduct rồi đấy. Nếu như bạn sử dụng SqlYog thì bạn vào table **Products** và vào mục **Stored Procs** sẽ thấy một thủ tục vừa được tạo:

### Gọi Stored Procedure

Tạo xong rồi bây giờ làm thế nào để gọi đến Store này? Đơn giản để gọi tới Store nào thì ta chỉ cần dùng cú pháp như sau:

```
1 CALL storeName();
```

Như vậy để gọi tới Procedure tên là GetAllProducts thì ta làm như sau:

```
1 CALL GetAllProducts();
```

### Xem danh sách Stored Procedure trong hệ thống

Nếu bạn không sử dụng SqlYog thì rất khó để quản lý Procedure vì không nhìn thấy được nó. Yên tâm vì trong MYSQL có hỗ trợ một số lệnh hiển thị danh sách Stored Procedure bằng cách chạy lệnh sau:

```
1 show procedure status;
```

### Sửa Stored Procedure đã tạo

Trong Mysql không cung cấp lệnh sửa Stored nên thông thường chúng ta sẽ chạy lệnh tạo mới. Tuy nhiên có một lưu ý đó là nếu như bạn đã chạy lệnh tạo Procedure một lần rồi, sau đó bạn edit và chạy lại thì ngay lập tức sẽ bị báo lỗi ngay vì trùng tên. Để giải quyết vấn đề này thì chúng ta sẽ dùng lệnh Drop để xóa đi Procedure đó và tạo lại:

```
1 DELIMITER $$
```

```
2
```

```
3 DROP PROCEDURE IF EXISTS `GetAllProducts`$$
```

```
4
```

```
5 CREATE PROCEDURE `GetAllProducts`()
```

```
6 BEGIN
```

```
7     SELECT * FROM products;
```

```
8      END$$
9
10     DELIMITER ;
```

## Bài 02: Biến (variable) trong MYSQL Stored Procedure

### 1. Khai báo biến trong MySql Stored Procedure

---

Để định nghĩa một biến mới ta dùng cú pháp :

```
1  DECLARE variable_name datatype(size) DEFAULT default_value
```

**Trong đó:**

- **DECLARE:** là từ khóa tạo biến
- **variable\_name** là tên biến
- **datatype(size)** là kiểu dữ liệu của biến và kích thước của nó
- **DEFAULT default\_value:** là gán giá trị mặc định cho biến

**Ví dụ:**

```
1  DECLARE product_title VARCHAR(255) DEFAULT 'No Name';
```

### Gán giá trị cho biến trong MySql Stored Procedure

---

Tạo biến rồi thì phải gán giá trị cho nó chứ đúng không nào? Để gán giá trị cho nó thì chúng ta sử dụng từ khóa SET:

```
1  SET variable_name = 'value';
```

**Ví dụ:** Định nghĩa biến age và gán giá trị 20 cho nó.

```
1  DECLARE age INT(11) DEFAULT 0
```

```
2
```

```
3  SET age = 12
```

**Ví dụ:** Gán giá trị thông qua lệnh SELECT

```
1  DECLARE total_products INT DEFAULT 0
```

```
2
```

```
3  SELECT COUNT(*) INTO total_products
4  FROM products
```

Trong ví dụ này thì trước tiên nó sẽ thực hiện câu truy vấn SQL đếm tổng số record và sau đó gán vào biến `total_products` bằng lệnh `(COUNT(*) INTO total_products)`.

#### 4. Phạm vi hoạt động của biến

---

Nếu như bạn định nghĩa một biến bên trong phần thân của Procedure (giữa *BEGIN* và *END*) thì đó ta gọi là biến cục bộ của Procedure. Bạn có thể định nghĩa nhiều biến trong một Procedure.

**Ví dụ:**

```
1
2  DELIMITER $$
3
4  DROP PROCEDURE IF EXISTS tinhTong $$
5
6  CREATE PROCEDURE tinhTong()
7
8  BEGIN
9
10     DECLARE a INT (11) DEFAULT 0;
11     DECLARE b INT (11) DEFAULT 0;
12     DECLARE tong INT (11) DEFAULT 0;
13
14     SET a = 200;
15     SET b = 300;
16     SET tong = a + b;
17
18     SELECT tong;
19
20 END; $$
21 DELIMITER;
```

Ở procedure này tôi đã định nghĩa các biến `a, b, tong` và tính toán trên đó. Bạn chạy câu sql trên và sau đó gọi nó bằng lệnh `call tinhTong` thì nó sẽ cho kết quả là 500.

Nếu một biến được khai báo bên ngoài Procedure thì bên trong Procedure sẽ không nhận được và sẽ thông báo lỗi.

**Ví dụ:**

```
1
    DELIMITER $$
2
    DECLARE tong INT (11) DEFAULT 0;
3
    DROP PROCEDURE IF EXISTS tinhTong $$
4
    CREATE PROCEDURE tinhTong()
5
    BEGIN
6
        DECLARE a INT (11) DEFAULT 0;
7
        DECLARE b INT (11) DEFAULT 0;
8
9
        SET a = 200;
10
        SET b = 300;
11
        SET tong = a + b;
12
13
        SELECT tong;
14
15
    END; $$
16
    DELIMITER;
```

Chương trình này lỗi vì biến `tong` không tồn tại trong Procedure.

## Bài 03: Truyền tham số vào Mysql Stored Procedure

Như ta biết thông thường một hàm sẽ có các tham số truyền vào và đối với ngôn ngữ lập trình thì sẽ tồn tại khái niệm tham chiếu và tham trị. Nhưng với Procedure trong MYSQL thì sẽ tồn tại ba loại đó là **tham số IN**, **tham số OUT** và **tham số INOUT** tuy nhiên về bản chất thì nó rất giống nhau. Chi tiết thế nào thì chúng ta tìm hiểu ở các phần dưới đây nhé.

Vậy thì trong bài này chúng ta sẽ tìm hiểu cách truyền một tham số (*variable*) vào một **Stored Procedure** như thế nào? Nhưng trước tiên chúng ta tìm hiểu cú pháp của nó đã nhé

## 2. Các loại tham số trong Mysql Stored Procedure

---

Chúng ta có hai loại tham số chính trong Procedure đó là:

- **IN**: Đây là chế độ mặc định (*nghĩa là nếu bạn không định nghĩa loại nào thì nó sẽ hiểu là IN*). Khi bạn sử dụng mức này thì nó sẽ được bảo vệ an toàn, có nghĩa là sẽ không bị thay đổi nếu như trong Procedure có tác động đến
- **OUT**: Chế độ này nếu như trong Procedure có tác động thay đổi thì nó sẽ thay đổi theo. Nhưng có điều đặc biệt là dù trước khi truyền vào mà bạn gán giá trị cho biến đó thì vẫn sẽ không nhận được vì mặc định nó luôn hiểu giá trị truyền vào là NULL.
- **INOUT**: Đây là sự kết hợp giữa **IN** và **OUT**. Nghĩa là **có thể gán giá trị trước và có thể bị thay đổi** nếu trong Procedure có tác động tới

**Ví dụ:**

```
1 DELIMITER $$
2 CREATE PROCEDURE getById(IN id INT(11))
3 BEGIN
4     /*Code*/
5 END; $$
6 DELIMITER;
```

Nếu muốn truyền vào nhiều hơn một tham số thì ta sẽ ngăn cách nó bởi dấu phẩy. Ví dụ:

```
1 DELIMITER $$
2 DROP PROCEDURE IF EXISTS getById $$
3 CREATE PROCEDURE getById(IN id INT(11), IN title VARCHAR(255))
4 BEGIN
5     /*Code*/
6 END; $$
7 DELIMITER;
```

7

Thông thường chúng ta viết các tham số xuống hàng để nhìn đẹp hơn. Ví dụ:

```
1      DELIMITER $$
2      DROP PROCEDURE IF EXISTS getById $$
3
4      CREATE PROCEDURE getById(
5          IN id INT(11),
6          IN title VARCHAR(255)
7      )
8      BEGIN
9          /*Code*/
10         END; $$
11         DELIMITER;
```

### 3. Tham số loại IN trong Mysql Stored Procedure

---

Như trình bày ở trên tham số này sẽ được bảo vệ và không bị thay đổi trong quá trình sử dụng trong Procedure.

**Ví dụ:** Viết Store lấy chi tiết sản phẩm theo ID

```
1      DELIMITER $$
2      DROP PROCEDURE IF EXISTS getById $$
3      CREATE PROCEDURE getById(IN idVal INT(11))
4      BEGIN
5          SELECT * FROM products WHERE id = idVal;
6      END; $$
7      DELIMITER;
```

Chạy Procedure này:

```
1 CALL getById(1);
```

Và ta có giao diện kết quả trả về:

|                          | id | title                                 | content                          |     |
|--------------------------|----|---------------------------------------|----------------------------------|-----|
| <input type="checkbox"/> | 1  | Học lập trình online tại freetuts.net | Gioi thieu website Học lập tr... | 63B |

#### 4. Tham số loại OUT trong Mysql Stored Procedure

---

Loại out nếu trong quá trình thực thi mà Procedure có tác động đến tham số này thì bên ngoài nó ảnh hưởng theo. Khi nhận tham số này thì Procedure sẽ hiểu đó là giá trị NULL nên dù bạn có gán giá trị cho biến trước khi truyền vào nó vẫn lấy NULL.

- Biến truyền vào phải có chữ @ đằng trước, ví dụ @title

**Ví dụ:** Truyền tham số title kiểu OUT vào Procedure và đổi giá trị cho nó, sau đó bên ngoài Procedure hiển thị giá trị của title.

```
1 DELIMITER $$
2 DROP PROCEDURE IF EXISTS changeTitle $$
3 CREATE PROCEDURE changeTitle(OUT title VARCHAR(255))
4 BEGIN
5     SET title = 'Hoc lap trinh online tai freetuts.net';
6 END; $$
7 DELIMITER;
```

Bây giờ ta gọi Procedure này như sau:

```
1 CALL changeTitle(@title);
2
3 SELECT @title;
```

Thì kết quả sẽ như sau:

|                          | @title                           |     |
|--------------------------|----------------------------------|-----|
| <input type="checkbox"/> | Hoc lap trinh online tai free... | 37B |

Như vậy ra rút ra kết luận như sau:



- Khi truyền tham số dạng `OUT` mục đích là lấy dữ liệu trong Procedure và sử dụng ở bên ngoài.
- Khi truyền tham số vào dạng `OUT` phải có chữ `@` đằng trước biến
- Hoạt động giống tham chiếu nên biến truyền vào dạng `OUT` không cần định nghĩa trước, chính vì vậy khởi đầu nó có giá trị `NULL`

## 5. Tham số dạng INOUT trong Mysql Stored Procedure

---

`INOUT` là sự kết hợp giữa `IN` và `OUT`, nghĩa là:

- Nó có thể được định nghĩa trước và gán giá trị trước rồi truyền vào Procedure, điều này với dạng `OUT` thì không thể được nhưng `IN` thì được.
- Sau khi thực thi xong nếu trong Procedure có tác động đến thì ảnh hưởng theo. Điều này dạng `IN` không được nhưng `OUT` thì không được.

**Ví dụ: Tạo Procedure**

```

1  DELIMITER $$
2
3  DROP PROCEDURE IF EXISTS counter $$
4
5  CREATE PROCEDURE counter (INOUT number INT(11))
6  BEGIN
7      SET number = number + 1;
8  END; $$
9  DELIMITER;
```

**Gọi sử dụng:**

```

1  SET @counter = 1;
2  CALL counter (@counter);
3  SELECT @counter;
```

Và kết quả là 2.

Nhưng nếu ta dùng dạng `OUT` thì kết quả sẽ là `NULL`. Lý do là bên trong **có tăng lên 1** nhưng nó lấy giá trị truyền vào dạng `OUT` là `NULL` nên `1 + NULL` sẽ là `NULL`.

## Bài 04: Câu lệnh if else trong MYSQL

### 1. Tìm hiểu mệnh đề if else trong MySql

---

Mệnh đề if cho phép bạn tạo luồng xử lý rẽ nhánh, nếu đúng thì thực thi và ngược lại mệnh đề sai thì nó sẽ không thực thi. Thông thường chúng ta kết hợp các toán tử, toán hạng và **biến trong mysql** để tạo ra các mệnh đề đúng sai trong điều kiện của lệnh `IF`. Không những chỉ có `IF` mà ta có thể sử dụng mệnh đề `IF ELSE` trong `MYSQL` cũng được.

**Cú pháp mệnh đề if - else trong MYSQL như sau:**

```
1  IF if_expression THEN
2      commands
3  ELSEIF elseif_expression THEN
4      commands
5  ELSE
6      commands
7  END IF;
```

**Luồng đi như sau:**

- Nếu `if_expression` đúng thì nó sẽ thực thi câu lệnh bên dưới nó, ngược lại nó bỏ qua và nhảy xuống `IFELSE`
- Nó kiểm tra mệnh đề `IFELSE`, nếu mệnh đề này đúng thì nó xử lệnh bên dưới, ngược lại thì nó bỏ qua và nhảy tiếp xuống dưới.
- Ở dưới nó nhận thấy chỉ còn có `ELSE` nên thực thi luôn chứ không cần kiểm tra điều kiện nữa.

Lưu ý với bạn là ta có thể có nhiều `IF ELSE` chứ không phải chỉ 1 cái như trong ví dụ trên.

### 2. Ví dụ mệnh đề if else trong MySql Stored Procedure

---

Bây giờ ta sẽ làm một ví dụ cho bạn dễ hiểu hơn. Trước tiên ta cần tạo một bảng thành viên và insert một số thông tin Username và Password. Bạn chạy lệnh sau để tạo bảng:

```
1  CREATE TABLE IF NOT EXISTS `members` (
2      `us_id` INT(11) NOT NULL AUTO_INCREMENT,
3      `us_username` VARCHAR(30) COLLATE utf8_unicode_ci DEFAULT NULL,
```

```

4      `us_password` VARCHAR(32) COLLATE utf8_unicode_ci DEFAULT NULL,
5      `us_level` TINYINT(1) DEFAULT '0',
6      PRIMARY KEY (`us_id`)
7  ) ENGINE=INNODB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci AUTO_INCREMENT=4 ;
8
9  --
10 -- Contenu de la table `members`
11 --
12 INSERT INTO `members` (`us_id`, `us_username`, `us_password`, `us_level`) VALUES
13 (1, 'admin', '57e34a1be668ebd6e40d430806beb099', 1),
14 (2, 'member', '57e34a1be668ebd6e40d430806beb099', 2),
15 (3, 'banded', '57e34a1be668ebd6e40d430806beb099', 0);
16

```

Trong bảng này ta cần chú ý đến field `us_level` như sau:

- Nếu `us_level = 0` => tài khoản bị khóa
- Nếu `us_level = 1` => admin
- Nếu `us_level = 2` => member

Bây giờ ta viết Procedure đăng nhập với yêu cầu như sau:

- Nếu `us_level = 0` => tài khoản bị khóa
- Nếu `us_level = 1` => là admin
- Nếu `us_level = 2` => là member
- Nếu không tồn tại => đăng nhập sai

**Ý tưởng:**

- Tạo Procedure với tham số truyền vào là gồm `username` và `password` thuộc loại `IN`, còn `result` thuộc loại `OUT` để lấy sử dụng. Nếu chưa biết hai khái niệm `IN` và `OUT` vui lòng đọc lại bài [tham số trong Procedure](#).

- Ta sẽ tạo một biến flag để lưu trữ `us_level` của người dùng, giá trị khởi tạo của nó là `-1`. Sau khi thực hiện lệnh `SELECT` nếu giá trị `flag = -1` tức là không tồn tại `username` và `password` trong CSDL, ngược lại thì ta sẽ `checkflag` để trả về kết quả tương ứng.

#### Bài giải:

```
1  DELIMITER $$
2
3  DROP PROCEDURE IF EXISTS `checkLogin`$$
4
5  CREATE PROCEDURE `checkLogin` (
6      IN input_username VARCHAR(255),
7      IN input_password VARCHAR(255),
8      OUT result VARCHAR(255)
9  )
10 BEGIN
11     /*Bien flag lưu trữ level. Mặc định là -1*/
12
13     DECLARE flag INT(11) DEFAULT -1;
14
15     /*Thực hiện truy vấn lấy level vào biến flag*/
16
17     SELECT us_level INTO flag FROM members
18     WHERE us_username = input_username AND us_password = MD5(input_password);
19
20     /*Sau khi thực hiện lệnh select này mà không có dữ liệu thì
21        lúc này flag sẽ không thay đổi. Chính vì thế nếu flag = -1 tức là sai thông tin
22        */
23
24     IF (flag <= 0) THEN
25         SET result = 'Thông tin đăng nhập sai';
```

```

21         ELSEIF (flag = 0) THEN
22             SET result = 'Tai khoan bi khoa';
23         ELSEIF (flag = 1) THEN
24             SET result = 'Tai khoan admin';
25         ELSE
26             SET result = 'Tai khoan member';
27         END IF;
28     END$$
29 DELIMITER ;
30
31
32

```

#### **Sử dụng:**

```

1
2     CALL checkLogin('admin', 'vancuong', @result);
3     SELECT @result;
4
5     -- hoặc
6
7     CALL checkLogin('member', 'vancuong', @result);
8     SELECT @result;
9     -- hoặc
10    CALL checkLogin('banded', 'vancuong', @result);
11    SELECT @result;
12

```

